

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

1. Selection sort,
2. Heapsort,
3. Smoothsort,
4. Cartesian tree sorting,
5. Cycle sort

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Selection sort е алгоритъм за сортиране на място за сравнение. Той има $O(n^2)$ времева сложност, което го прави не високо ефективен при големи списъци.

Сортирането чрез избор се отличава със своята простота и има предимства в производителността пред по-сложните алгоритми в определени ситуации, особено когато паметта е ограничена.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Алгоритъмът разделя входния списък на две части: сортиран подсписък от елементи, който е изграден отляво надясно в предната (лява) част на списъка и подсписък на останалите несортирани елементи, които заемат останалата част от списъка. Първоначално сортираният подсписък е празен, а несортираният подсписък е целият входен списък.

Алгоритъмът продължава, като намира най-малкия (или най-големия, в зависимост от реда на сортиране) елемент в несортирания подсписък, разменя го с най-левия несортиран елемент (поставя го в сортиран ред) и премества границите на подсписъка с един елемент надясно.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Времевата ефективност на selection sort е $O(n^2)$ квадратична, така че има редица техники за сортиране, които имат по-добра времева сложност от него.

Selection sort

Сортиран подписък	Несортиран подписък	Най-малък несортиран елемен
()	(11, 25, 12, 22, 64)	11
(11)	(25, 12, 22, 64)	12
(11, 12)	(25, 22, 64)	22
(11, 12, 22)	(25, 64)	25
(11, 12, 22, 25)	(64)	64
(11, 12, 22, 25, 64)	()	

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

```
// Selection sort in C++
```

```
#include <iostream>
```

```
using namespace std;
```

```
// function to swap the the position of two elements
```

```
void swap(int *a, int *b) {
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// function to print an array
```

```
void printArray(int array[], int size) {
```

```
    for (int i = 0; i < size; i++) {
```

```
        cout << array[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
void selectionSort(int array[], int size) {
```

```
    for (int step = 0; step < size - 1; step++) {
```

```
        int min_idx = step;
```

```
        for (int i = step + 1; i < size; i++) {
```

```
            // To sort in descending order, change > to < in this line.
```

```
            // Select the minimum element in each loop.
```

```
            if (array[i] < array[min_idx])
```

```
                min_idx = i;
```

```
        }
```

```
        // put min at the correct position
```

```
        swap(&array[min_idx], &array[step]);
```

```
    }
```

```
}
```

```
// driver code
```

```
int main() {
```

```
    int data[] = {20, 12, 10, 15, 2};
```

```
    int size = sizeof(data) / sizeof(data[0]);
```

```
    selectionSort(data, size);
```

```
    cout << "Sorted array in Ascending Order:\n";
```

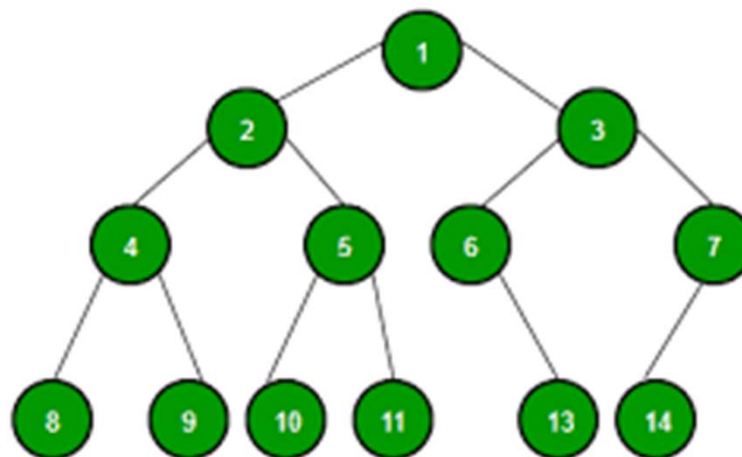
```
    printArray(data, size);
```

```
    Return 0;
```

```
}
```

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Структура от данни „дърво“ (Tree): Дървото в програмирането е рекурсивна структура от данни, която се състои от върхове, които са свързани помежду си с ребра. За дърветата са в сила твърденията: Всеки връх може да има “N” или повече преки наследници. Всеки връх има най-много един родител.



Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

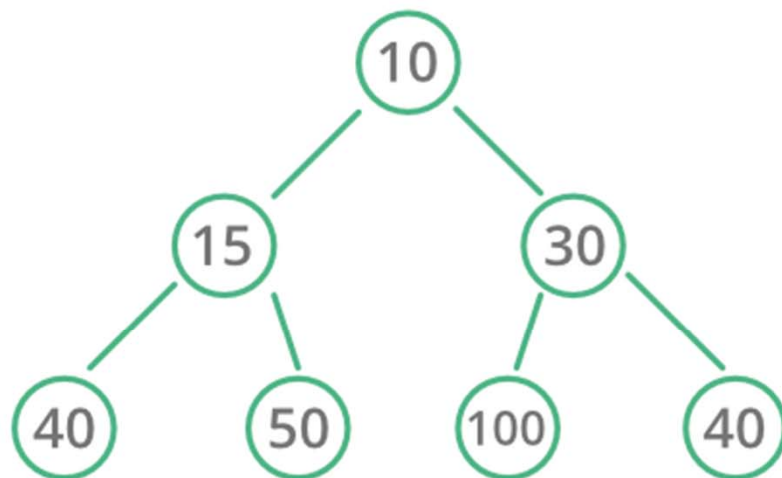
Heapsort (hear – купчина, група) дефиниция: Всеки възел в дървото има ключ, който е по-краен (по-голям или по-малък) от или равен на ключа на своя родител.

Свойства на Hear

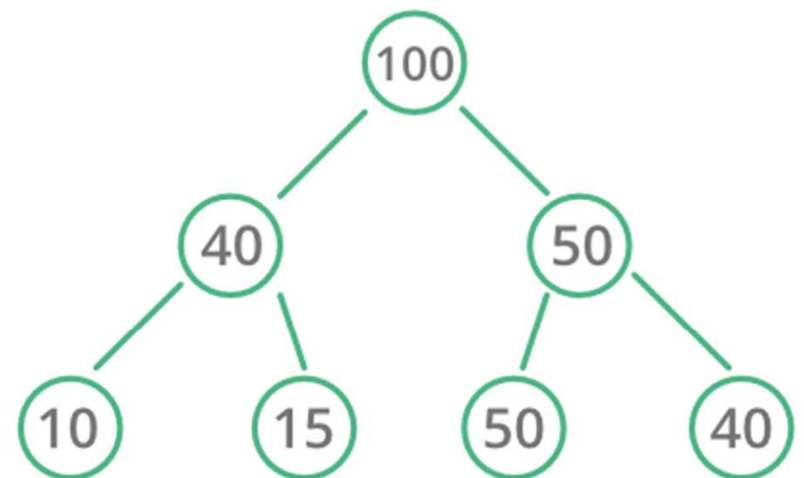
1. **Подредба:** Възлите трябва да бъдат подредени в ред според стойностите. Стойностите трябва да следват свойството min-hear или max-hear.
2. **Структура:** Всички нива в **Hear** трябва да са пълни.
3. **Методи или операции на Hear:** find - за да намерите елемент в Hear.
4. **Изпълнение:** Hear обикновено се реализира в масив.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Heap Data Structure



Min Heap



Max Heap

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Heapsort (hear – купчина; групово сортиране) може да се разглежда като подобрен Selection sort. Подобно на Selection sort, Heapsort разделя своя вход на сортиран и несортиран списък и итеративно свива несортирания списък, като извлича най-големия елемент от него и го вмъква в сортирания списък.

За разлика от Selection sort, **Heapsort** не губи време с сканиране в линейно време на несортираната област; по-скоро **Heapsort** поддържа несортираната област в структура от данни на hear, за да намери по-бързо най-големия елемент във всяка стъпка.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

```
// main function to do heap sort
void heapSort(int arr[], int n)
{
    // Build heap (rearrange array)
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // One by one extract an element from heap
    for (int i = n - 1; i >= 0; i--) {
        // Move current root to end
        swap(arr[0], arr[i]);

        // call max heapify on the reduced heap
        heapify(arr, i, 0);
    }
}
```

Heapsort

Производителността на този алгоритъм е $O(n + n \log n) = O(n \log n)$.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

```
// which is an index in arr[].
```

```
// n is size of heap
```

```
void heapify(int arr[], int N, int i)
```

```
{
```

```
    // Initialize largest as root
```

```
    int largest = i;
```

```
    // left = 2*i + 1
```

```
    int l = 2 * i + 1;
```

```
    // right = 2*i + 2
```

```
    int r = 2 * i + 2;
```

```
    // If left child is larger than root
```

```
    if (l < N && arr[l] > arr[largest])
```

```
        largest = l;
```

```
    // If right child is larger than largest
```

```
    // so far
```

```
    if (r < N && arr[r] > arr[largest])
```

```
        largest = r;
```

```
    // If largest is not root
```

```
    if (largest != i) {
```

```
        swap(arr[i], arr[largest]);
```

```
        // Recursively heapify the affected
```

```
        // sub-tree
```

```
        heapify(arr, N, largest);
```

```
    }
```

```
}
```

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Smoothsort е алгоритъм за сортиране, базиран на сравнение. Подобно на **heapsort**, **smoothsort** е алгоритъм с времева сложност $O(n \log n)$, като той не е алгоритъм за стабилно сортиране.

Предимството на **smoothsort** е, че се доближава до $O(n)$ по сложност по време, ако входът вече е сортиран до известна степен, докато **heapsort** осреднява $O(n \log n)$ независимо от първоначалното сортирано състояние.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Smooth sort (гладко сортиране) е подобрена версия на Heap sort (групово сортиране). Предимството е, че **Smooth sort** отнема $O(N)$ време с почти вече сортирани данни, а когато степента на безпорядък нараства, преходът от $O(N)$ към $O(N \log N)$ е плавен (оттук и името). Тоест **Smooth sort** отнема почти $O(N)$, когато данните са почти сортирани.

[Smooth sort](#)

[Smooth sort](#)

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Cartesian tree sorting (Сортирането по декартово дърво), наричано още алгоритъм на Левкопулос Петерсон, е адаптивен алгоритъм за сортиране, т.е. той се представя много по-добре при частично сортирани данни.

Необходими са две структури от данни: декартово дърво и приоритетна опашка. Алгоритъмът тук използва декартово дърво с min-heap, за да даде сортирана последователност.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Декартово дърво за поредица от числа се определя от следните свойства:

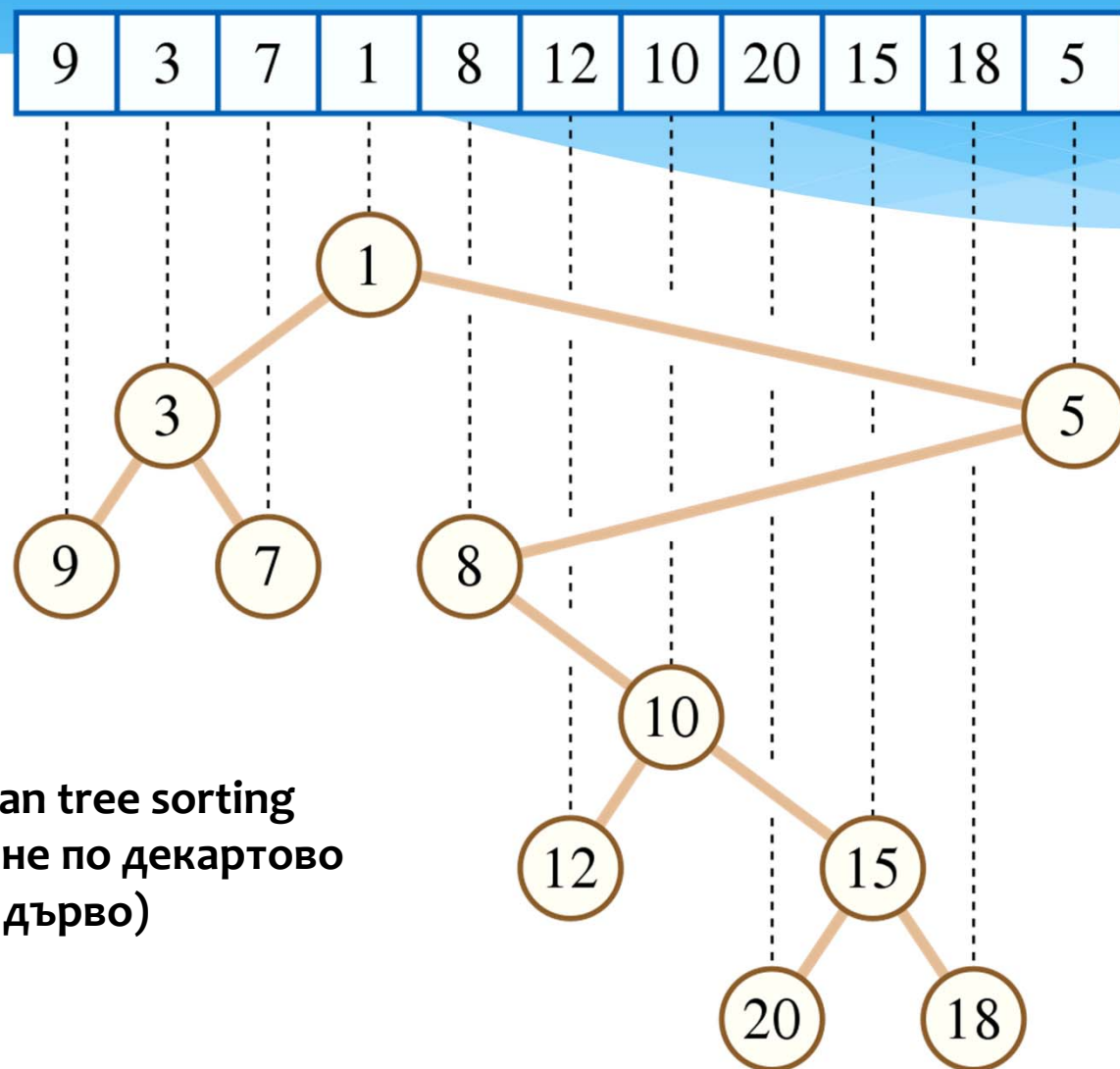
- Декартовото дърво за поредица от числа има един възел за всяко число в последователността. Всеки възел е свързан с една стойност на поредицата.
- Симетрично (в ред) обхождане на дървото води до оригиналната последователност. Това означава, че лявото поддърво се състои от стойностите, по-ранни от корена в реда на последователността, докато дясното поддърво се състои от стойностите, по-късни от корена, и подобно ограничение за подреждане се прилага за всеки по-нисък възел на дървото.
- Дървото има свойството heap: родителят на всеки различен от root възел има по-малка стойност от самия възел.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Въз основа на свойството **heap**, коренът на дървото трябва да бъде най-малкото число в последователността. От това самото дърво може също да бъде дефинирано рекурсивно: коренът е минималната стойност на последователността, а лявото и дясното поддърво са декартовите дървета за подпоследователностите отляво и отдясно на корена.

За поредица от различни числа трите свойства определят уникално декартово дърво. Ако поредица от числа съдържа повторения, декартовото дърво може да бъде определено чрез следване на последователно правило за прекъсване на равенството, преди да се приложи горната конструкция. Например, първият от два равни елемента може да се третира като по-малкия от двата.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)



Cartesian tree sorting
(Сортиране по декартово
дърво)

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Алгоритъмът на **Cartesian tree sorting** (Сортиране по декартово дърво) се състои от следните стъпки:

1. Конструирание на декартово дърво за входната последователност от данни
2. Инициализиране на приоритетна опашка, първоначално съдържаща само корена на дървото
3. Докато приоритетната опашка не е празна:
 - * Намера се и се премахва минималната стойност в опашката с приоритети
 - * Откритата и премахната стойност се добавя към изходната последователност
 - * Добавят се наследниците от декартово дърво на премахнатата стойност към опашката с приоритети

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

При Cartesian tree sorting (Сортиране по декартово дърво) за входни последователности, които вече са почти сортирани, размерът на приоритетната опашка ще остане малък, позволявайки на този метод да се възползва от почти сортирания вход и да работи по-бързо.

Времевата сложност на този метод за сортиране в най-лошия случай е **логаритмична**.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Цикълното сортиране (Cycle sort) е нестабилен алгоритъм за сортиране на място, сортиране със сравнение, което е теоретично оптимално по отношение на общия брой записи в оригиналния масив, за разлика от всеки друг алгоритъм за сортиране на място.

За разлика от почти всяко друго сортиране, елементите никога не се записват на друго място в масива, просто за да бъдат изместени от пътя на действието. Всяка стойност се записва или нула пъти, ако вече е в правилната си позиция, или се записва веднъж в правилната си позиция. Това съответства на минималния брой презаписвания, необходими за завършено сортиране на място.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Цикълното сортиране (Cycle sort) се основава на идеята, че списъкът, който трябва да бъде сортиран, може да бъде разложен на цикли, които могат да бъдат завъртани поотделно, за да дадат сортиран резултат.

Минимизирането на броя на записванията е полезно, когато извършването на записи в някакъв огромен набор от данни е много скъпо, като например с EEPROM като Flash памет, където всяко записване намалява продължителността на живота на паметта.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

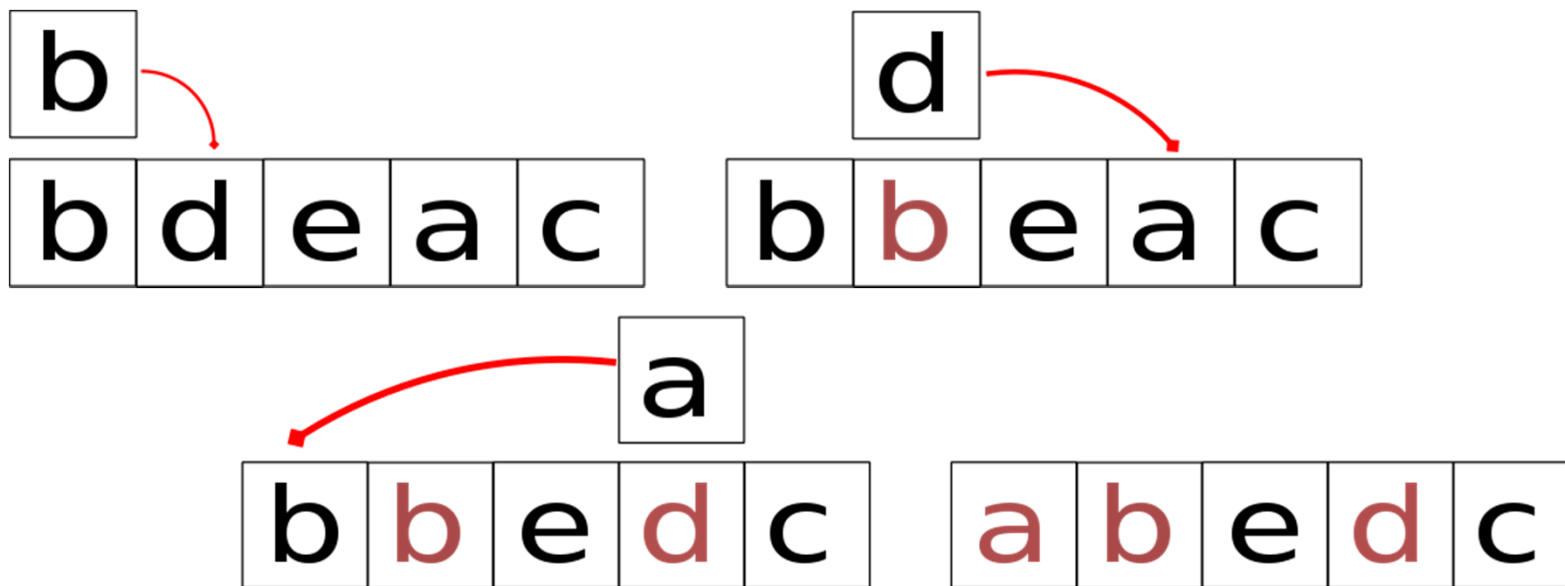
Алгоритъмът на цикълното сортиране (Cycle sort) може да се илюстрира по следния начин:

Даден е елемент Y , можем да намерим индекса, при който той ще се появи в сортирания списък, като просто преброим броя на елементите в целия списък, които са по-малки от Y .

- * Ако елементът вече е на правилната позиция, не прави нищо.
- * Ако не е, той се записва на предвидената му позиция. Тази позиция се заема от различен елемент Z , който след това трябва да преместим в правилната му позиция. Този процес на изместване на елементи до техните правилни позиции продължава, докато даден елемент не бъде преместен в първоначалната позиция на Y . Това завършва цикъла.

Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Повтарянето на този процес за всеки елемент сортира списъка с една операция за запис, ако и само ако даден елемент вече не е на правилната си позиция. Докато изчисляването на правилните позиции отнема $O(n)$ време за всеки отделен елемент, което води до алгоритъм с квадратична времева сложност $O(n^2)$, но броят на операциите за запис е сведен до минимум.



Тема 8 – Алгоритми за сортиране чрез избор (*Selection sorts*)

Псевдокод

Begin

for

start:= 0 to n - 2 do

key := array[start]

location := start

for i:= start + 1 to n-1 do

if array[i] < key then

location:=location +1

done

if location = start then

ignore lower part, go for next iteration

while key = array[location] do

location = location

done

if location != start then

swap array[location] with key

while location != start do

location start

for i:= start + 1 to n-1 do

if array[i] < key then

location:=location +1

done

while key= array[location]

location := location +1

if key != array[location]

Swap array[location] and key

done

done

End

Cycle Sort